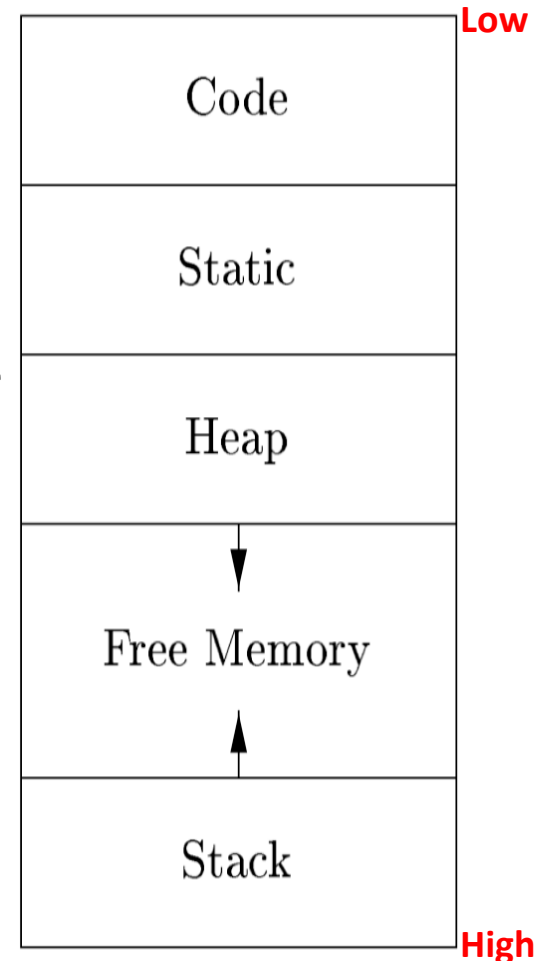


Activation Records

Chapter 6

Storage Organization

- From the compiler writer's perspective, the executing target program runs in its own **logical address space** in which each program value has a location.
- The run time representation of an object program consists of **data** and **program** areas.
- A typical way of subdividing memory is:
 - **Code**: the executable target code
 - **Static**: the data objects of which the size can be known at compile time, e.g.,
 - global constants
 - data generated by the compiler
 - **Stack**: data structures called **activation records** that get generated during procedure calls
 - **Heap**: data that is allocated and deallocated under program control
 - for C: malloc, free



Activation Records

```
function f(x: int): int =  
  let var y := x + x  
  in if y < 10  
    then f(y)  
    else return y - 1  
end
```

- There are recursive calls, many of these *x*'s exist simultaneously.

- An invocation of function *f* is an **activation** of *f*
- A new instantiation of *x* is **created** (and initialized by *f*'s caller) **each time that *f* is called**.
- In many languages (including C, Pascal, and Java), local variables are **destroyed** when **a function returns**.
- **How to hold local variables?**
 - Function calls behave in **last-in-first-out (LIFO)** fashion
 - Use a LIFO data structure – a **stack**

Activation Records

- Procedure calls and returns are usually managed by a run time stack called *control stack*.
- Each time a *procedure is called*, space for its local variables is *pushed onto the stack*.
- When the *procedure terminates* that space is *popped off the stack*.
- Procedure calls are also called *activations of procedures*.
- Each *live activation* has an *activation record* (sometimes called a *frame*) on the control stack.

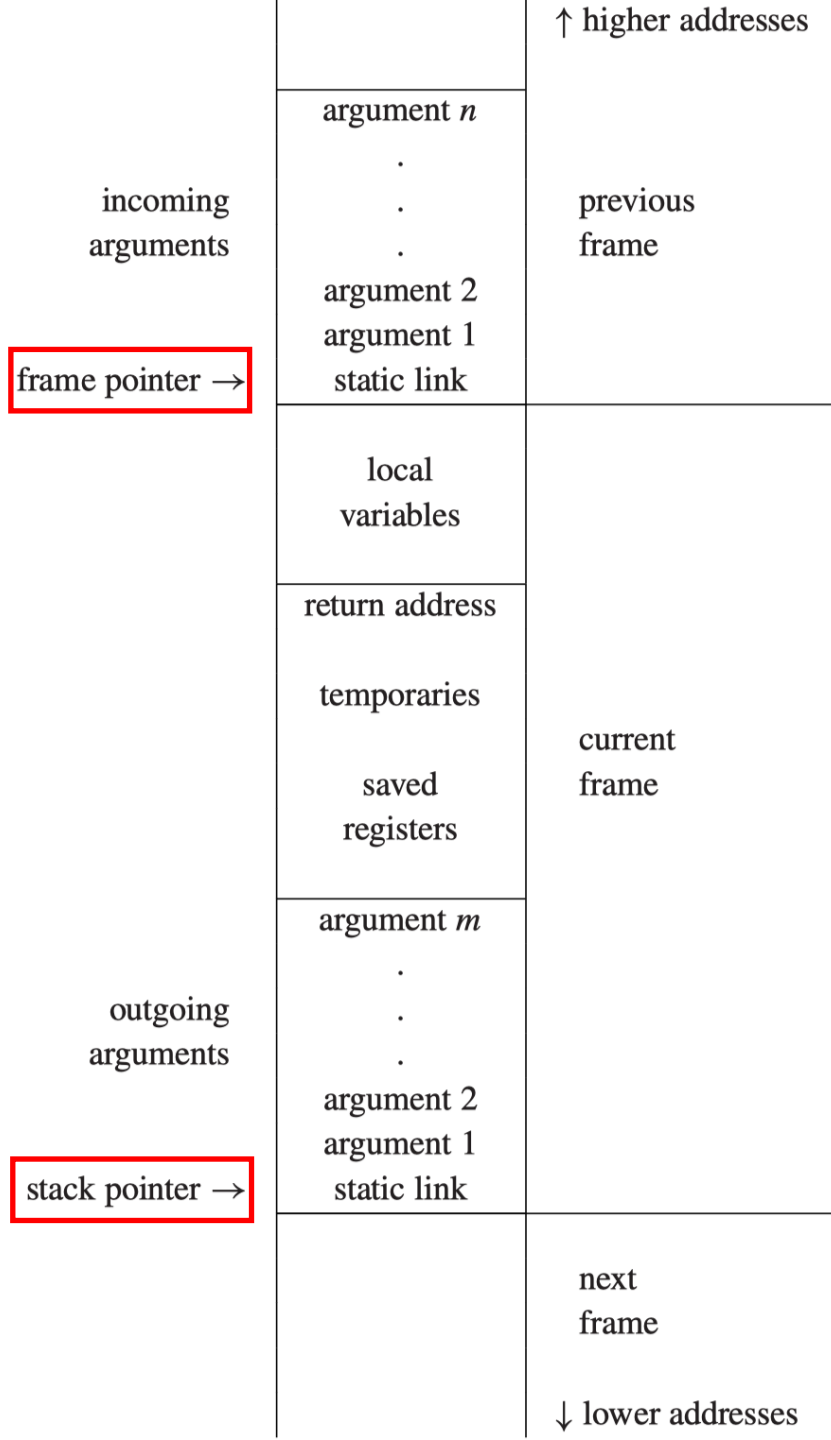
6.1 Stack Frames

6.1 Stack Frames

- Stack: **push** and **pop**
- However:
 - Local variables are pushed/popped in large batches
 - Local variables are not always initialized right after their creation
 - We want to continue accessing variables deep within the stack
- We treat the stack as a big array.
- **Stack Pointer** – a special register that points at some location
 - All locations **beyond the stack pointer** are considered to be **garbage**,
 - and all locations **before the stack pointer** are considered to **be allocated**.

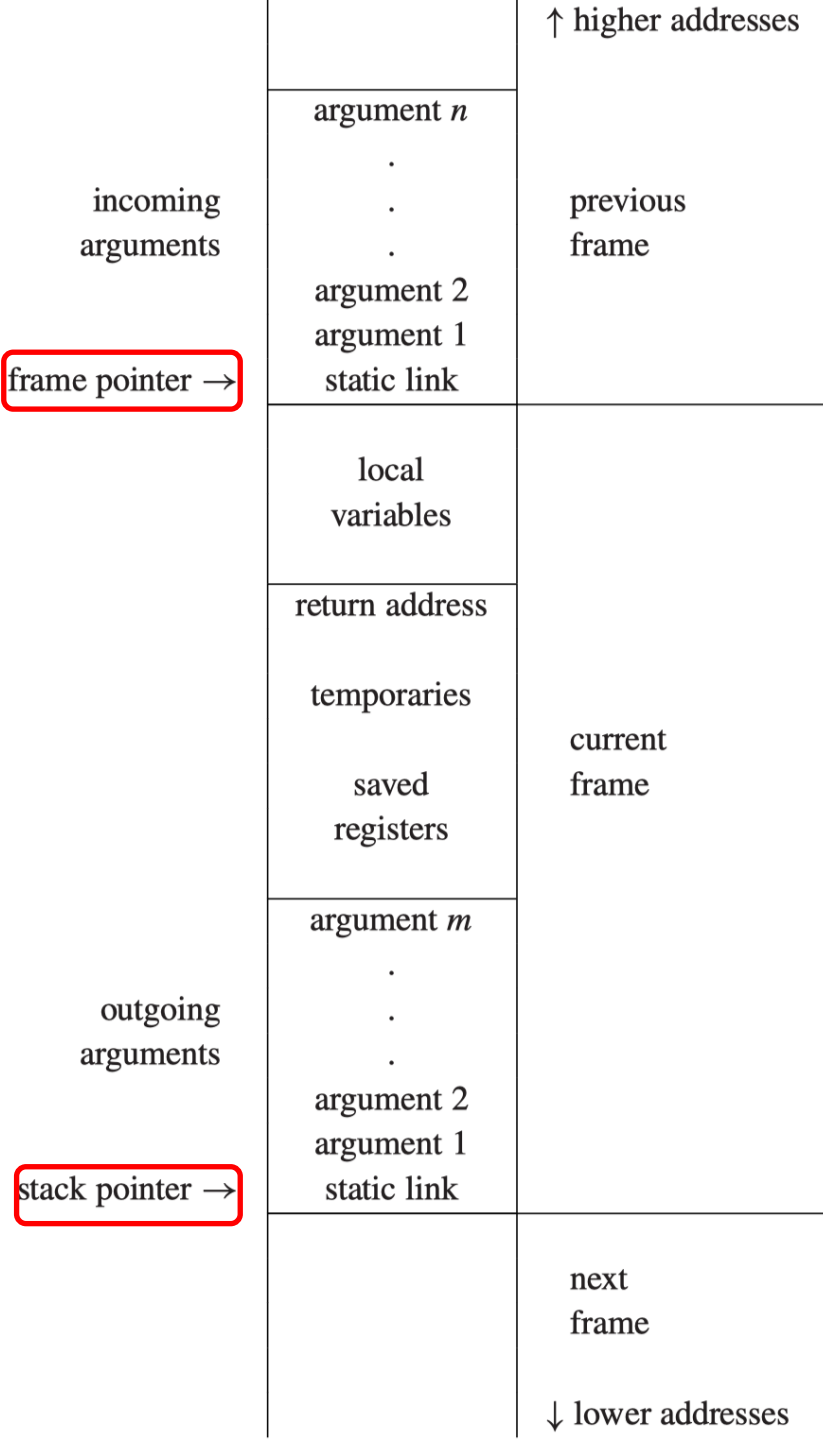
Stack Frame

- The stack usually **grows** only **at the entry to a function**, by an increment large enough to hold all the local variables for that function, and, just before **the exit from the function**, **shrinks** by the same amount.
- A function's **activation record** or **stack frame**: The area on the stack devoted to the local variables, parameters, return address, and other temporaries for this function.
- Run-time stacks usually start at a high memory address and grow toward smaller addresses.
- The stack frame connects the caller to the callee.
- **Main Problem**: how to **layout** the activation record so that the caller and callee can **communicate** properly?



A typical stack frame layout

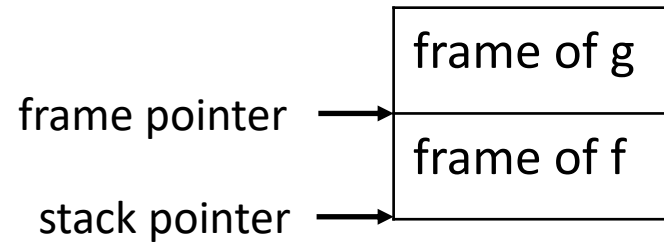
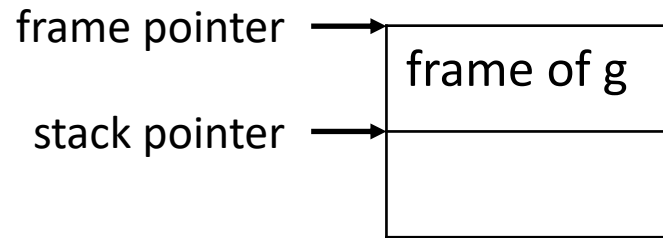
- *incoming arguments*: passed by the caller
- *local variables*: some are in this frame, others are kept in machine registers
- *the return address*: where (within the calling function) control should return:
 - created by the CALL instruction
- *temporaries*
- *saved registers*: make room for other uses of the registers
- *outgoing argument*: pass parameters to other functions
- *static link*



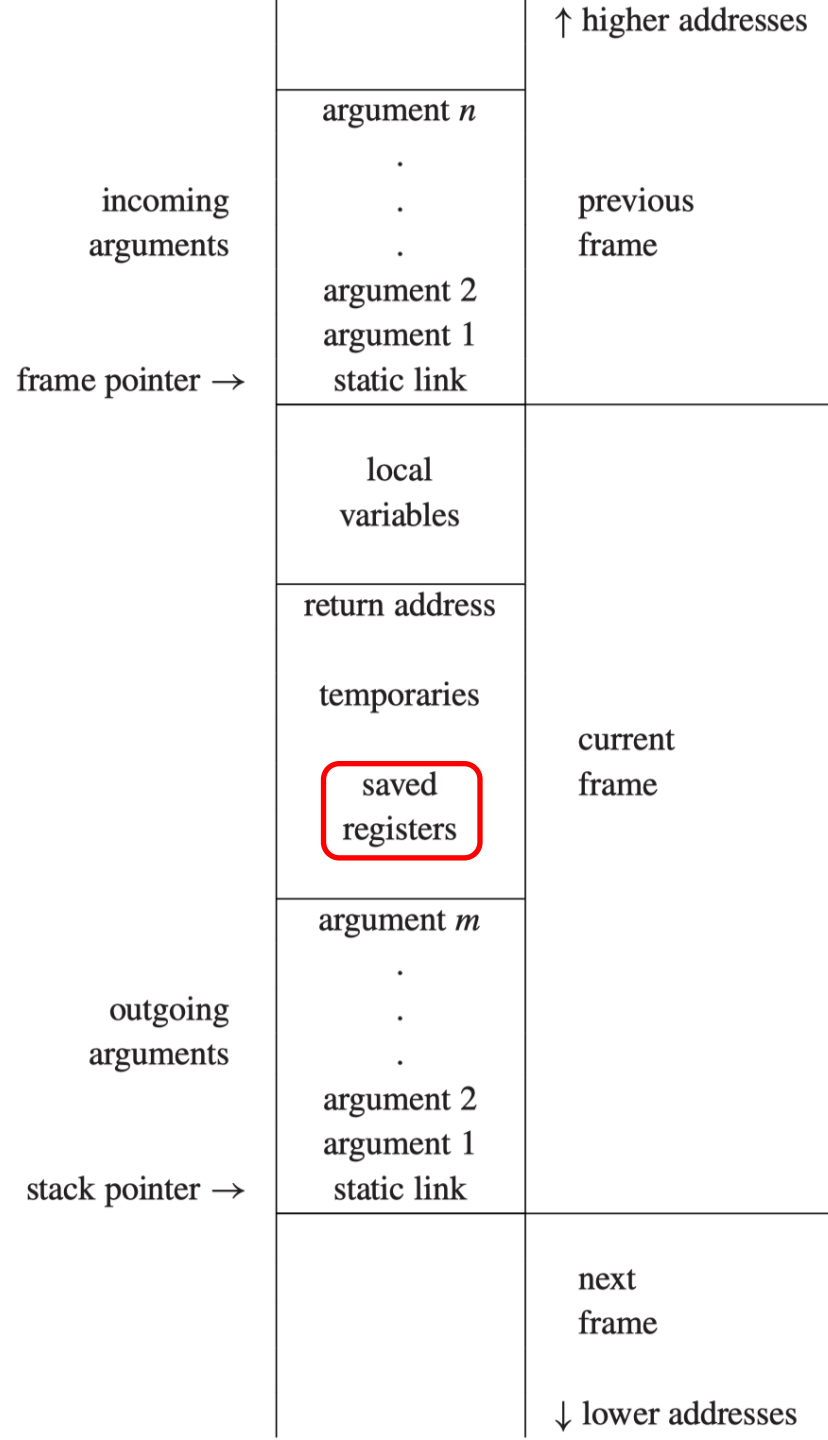
Frame Pointer

- Suppose a function $g(\dots)$ calls the function $f(a_1, \dots, a_n)$
 - g : caller
 - f : callee
- When g calls f :
 - The **stack pointer (SP)** points to the first argument that g passes to f
 - f allocates a frame by simply subtracting the frame size from the stack pointer **SP**

Frame Pointer

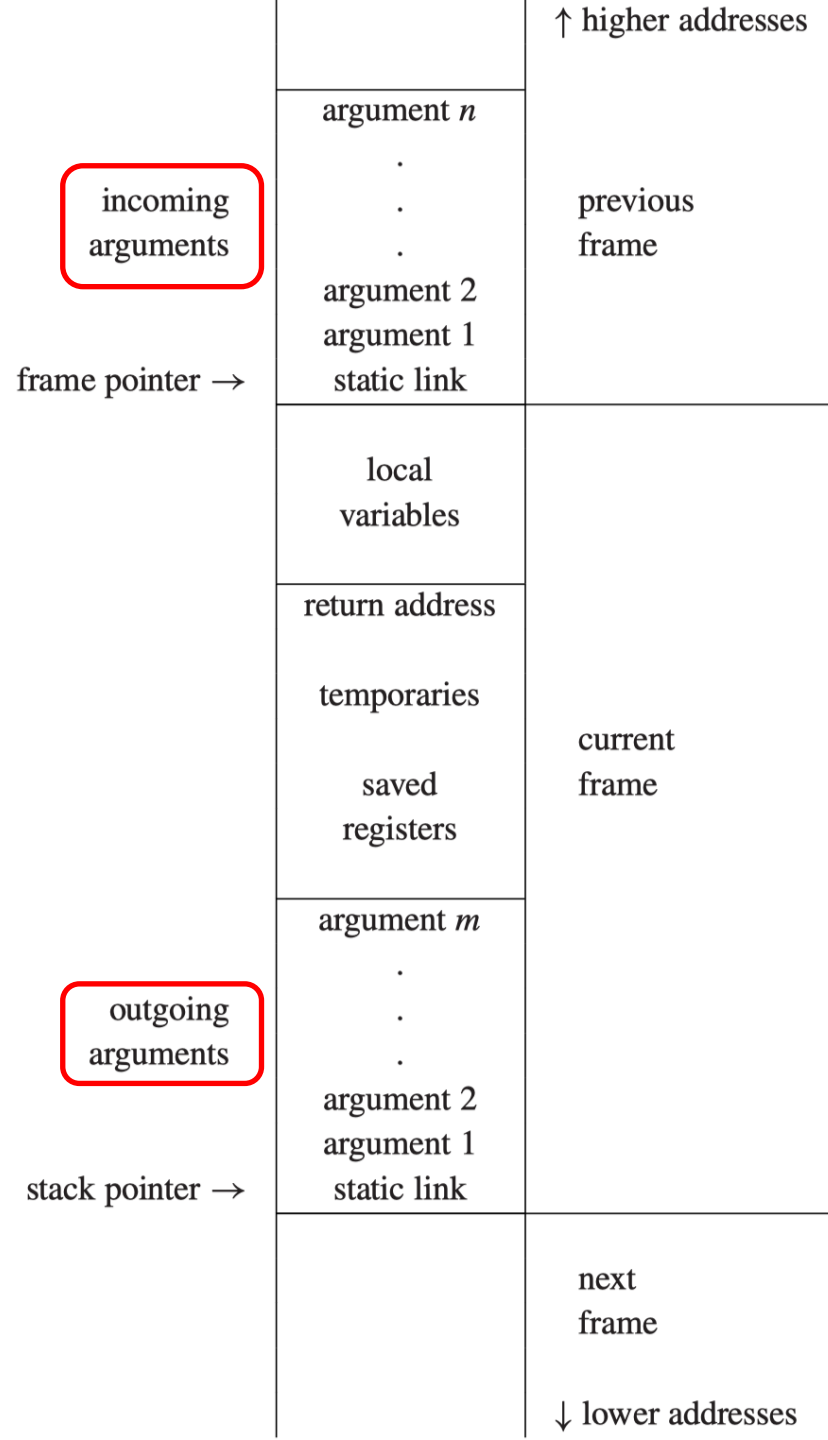


- When **enter f**:
 - save the old **frame pointer FP** in memory in the frame
 - $FP = SP$
 - $SP = SP - framesize$
- When **f exits**:
 - $SP = FP$
 - fetch back the saved old FP
- This arrangement is useful if **f**'s frame size can vary, or if frames are not always contiguous on the stack.
- If the frame size is fixed: $FP = SP + framesize$
 - FP is a “fictional” register



Registers

- Accessing registers is faster than accessing memory.
- A modern machine has a large set of *registers* (typically 32). But many different procedures and functions need to use registers.
- Suppose:
 - *f* is using register *r* to hold a local variable and calls procedure *g*.
 - *g* also uses *r* for its own calculations.
 - *r* must be **saved** (stored into a stack frame) before *g* uses it. and **restored** (fetched back from the frame) after *g* is finished using it.
- *r* is a **caller-save register** if the caller (in this case, *f*) must save and restore it.
- *r* is a **callee-save register** if it is the responsibility of the callee (in this case, *g*).
 - e.g., the register that saves the frame pointer.
- We will discuss how to allocate registers in Section 11.



Parameter Passing

- **Tiger:** **Pass-by-Value**
 - Values of the actual arguments are passed and established as values of formal parameters.
 - Modification to formals have no effect on actuals
- **Problem:** It causes needless memory traffic if arguments are passed only on the stack.
- **Solution:** **Parameter-passing conventions** for modern machines specify:
 - the **first k arguments** (for $k = 4$ or $k = 6$, typically) of a function are passed **in registers** $r_p, \dots, r_{p+k-1}$, and **the rest** of the arguments are passed **in memory**.

Parameter Passing

```
void f(int a) {  
    int z = ...  
    h(z);  
    ...  
    int t = a + 2;  
    ...  
}
```

- Suppose f received its parameter in register r_1 and calls $h(z)$
- f must pass z in r_1 .
 - f should save the old contents of r_1 in its stack frame before calling h
- Such memory traffic **was supposedly avoided** by passing arguments in registers.

Parameter Passing

```
void f(int a) {  
    int z = ...  
    h(z);  
    ...  
    int t = a + 2;  
    ...  
}
```

- Suppose f received its parameters in register r_1 and calls $h(z)$
- f must pass z in r_1 .
 - f should save the old contents of r_1 in its stack frame before calling h
- **How to avoid the extra memory traffic?**

1. Parameter a is a dead variable at the point where $h(z)$ is called. Then f can overwrite r_1 without saving it.

Parameter Passing

```
void f(int a) {  
    int z = ...  
    h(z);  
    ...  
    int t = a + 2;  
    ...  
}
```

- Suppose f received its parameters in register r_1 and calls $h(z)$
- f must pass z in r_1 .
 - f should save the old contents of r_1 in its stack frame before calling h
- **How to avoid the extra memory traffic?**

2. Leaf procedures need not write their incoming arguments to memory

leaf procedures: the procedures that don't call other procedures

Parameter Passing

```
void f(int a) {  
    int z = ...  
    h(z);  
    ...  
    int t = a + 2;  
    ...  
}
```

- Suppose f received its parameters in register r_1 and calls $h(z)$
- f must pass z in r_1 .
 - f should save the old contents of r_1 in its stack frame before calling h
- **How to avoid the extra memory traffic?**

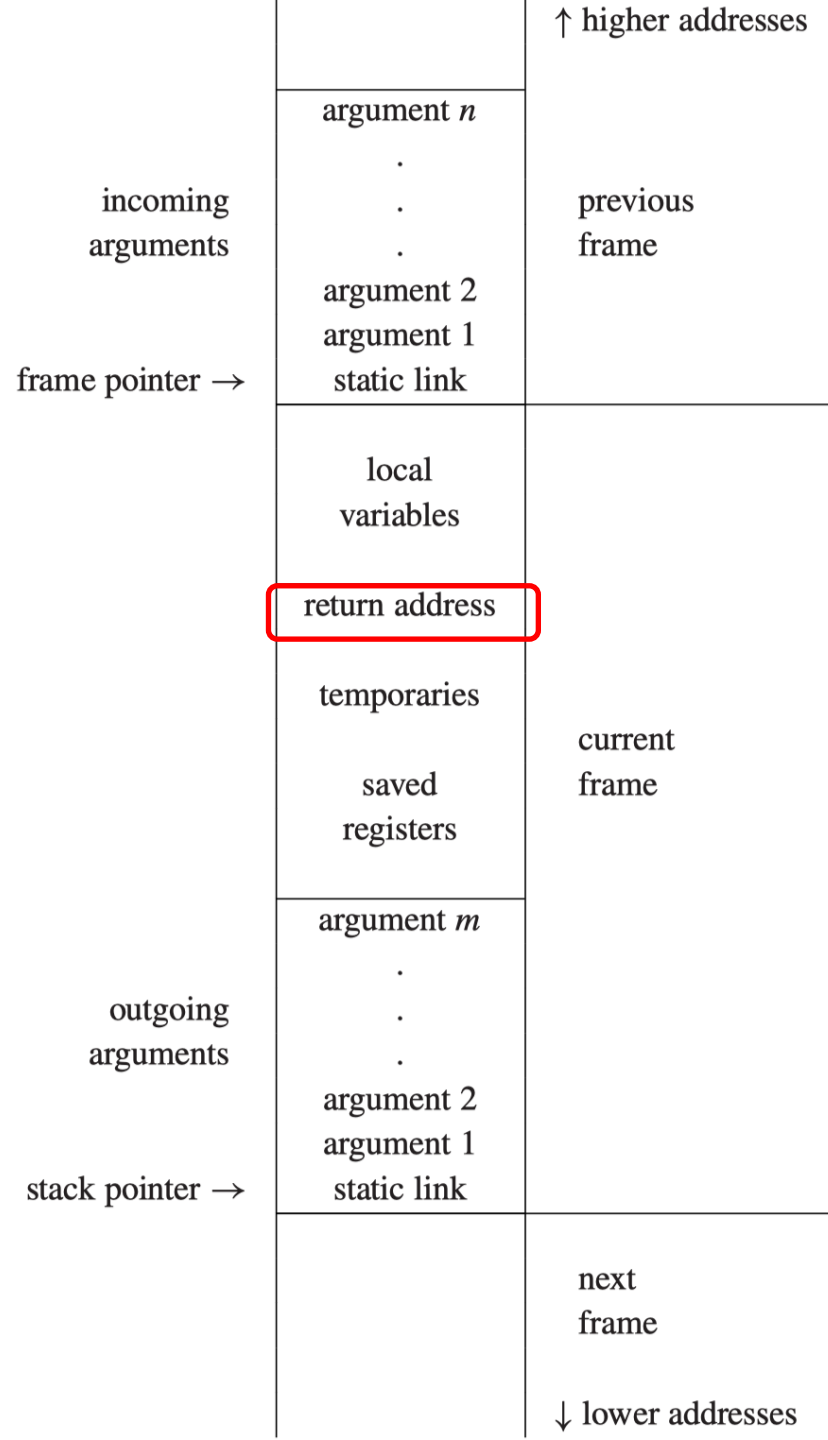
3. Some optimizing compilers use *inter-procedural register allocation*, analyzing all the functions in an entire program at once. They assign different procedures different registers to receive parameters and hold local variables

Parameter Passing

```
void f(int a) {  
    int z = ...  
    h(z);  
    ...  
    int t = a + 2;  
    ...  
}
```

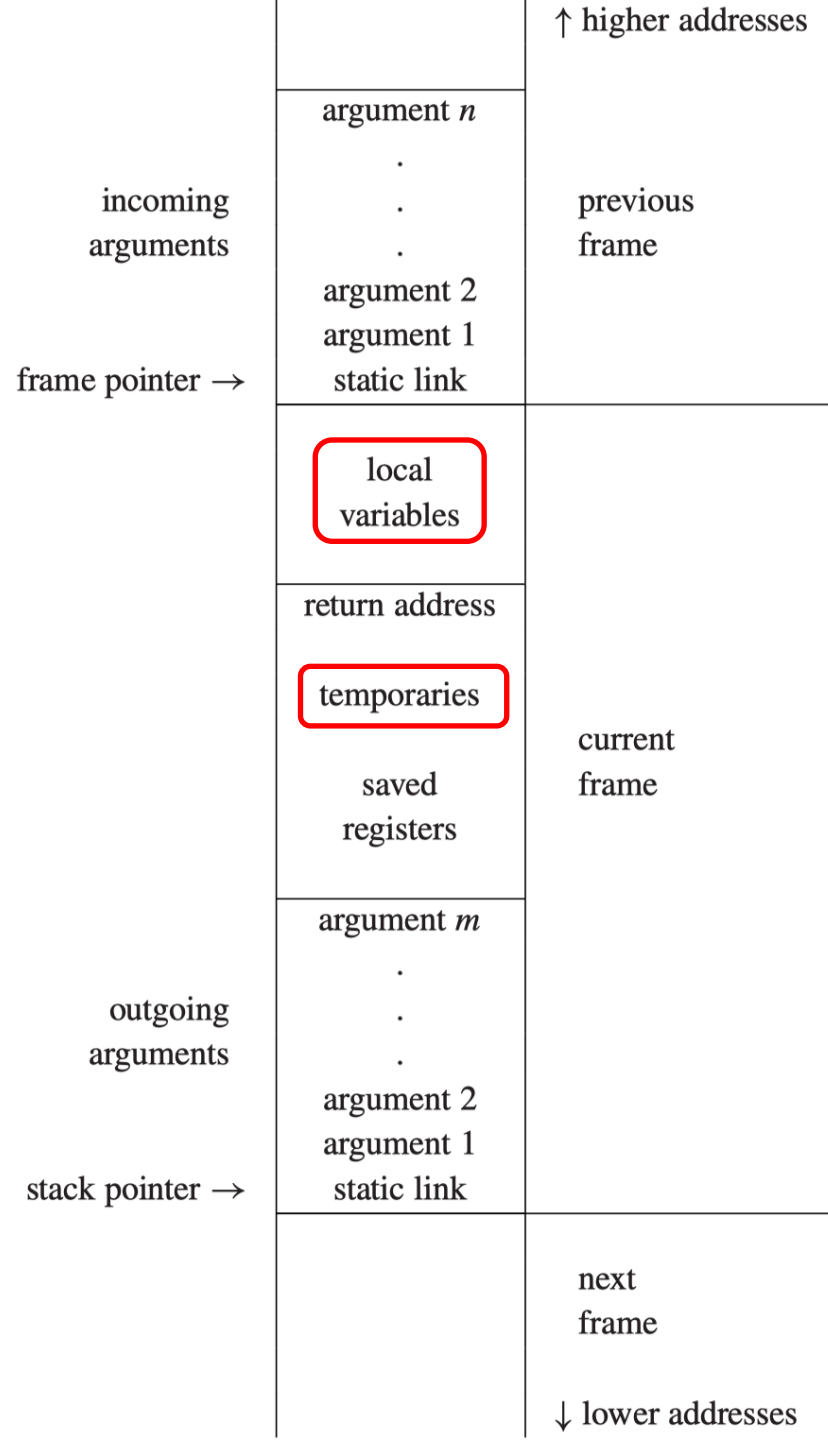
- Suppose f received its parameters in register r_1 and calls $h(z)$
- f must pass z in r_1 .
 - f should save the old contents of r_1 in its stack frame before calling h
- **How to avoid the extra memory traffic?**

4. Some architectures have **register windows**, so that each function invocation can allocate a fresh set of registers without memory traffic.



Return Address

- Suppose function g calls function f . When f returns, where to go back to?
- If the *call* instruction within g is at address a , then (usually) the right place to return to is $a + 1$, the next instruction in g .
- This is called the *return address*.
- On modern machines, the *call* instruction merely puts the return address in a *designated register*.
 - For MIPS, this register is $\$ra$
- A *non-leaf procedure* will have to write it to the stack
 - unless *inter-procedural register allocation* is used
- A *leaf procedure* will not.



Frame-Resident Variables

- Modern procedure-call convention will:
 - pass function parameters in registers
 - pass the return address in a register
 - return the function result in a register
- Many of local variables and intermediate results will be allocated to registers
- When to write values to memory (in the stack frame)?

Frame-Resident Variables

Values are written to memory (in the stack frame) only when necessary for one of these reasons:

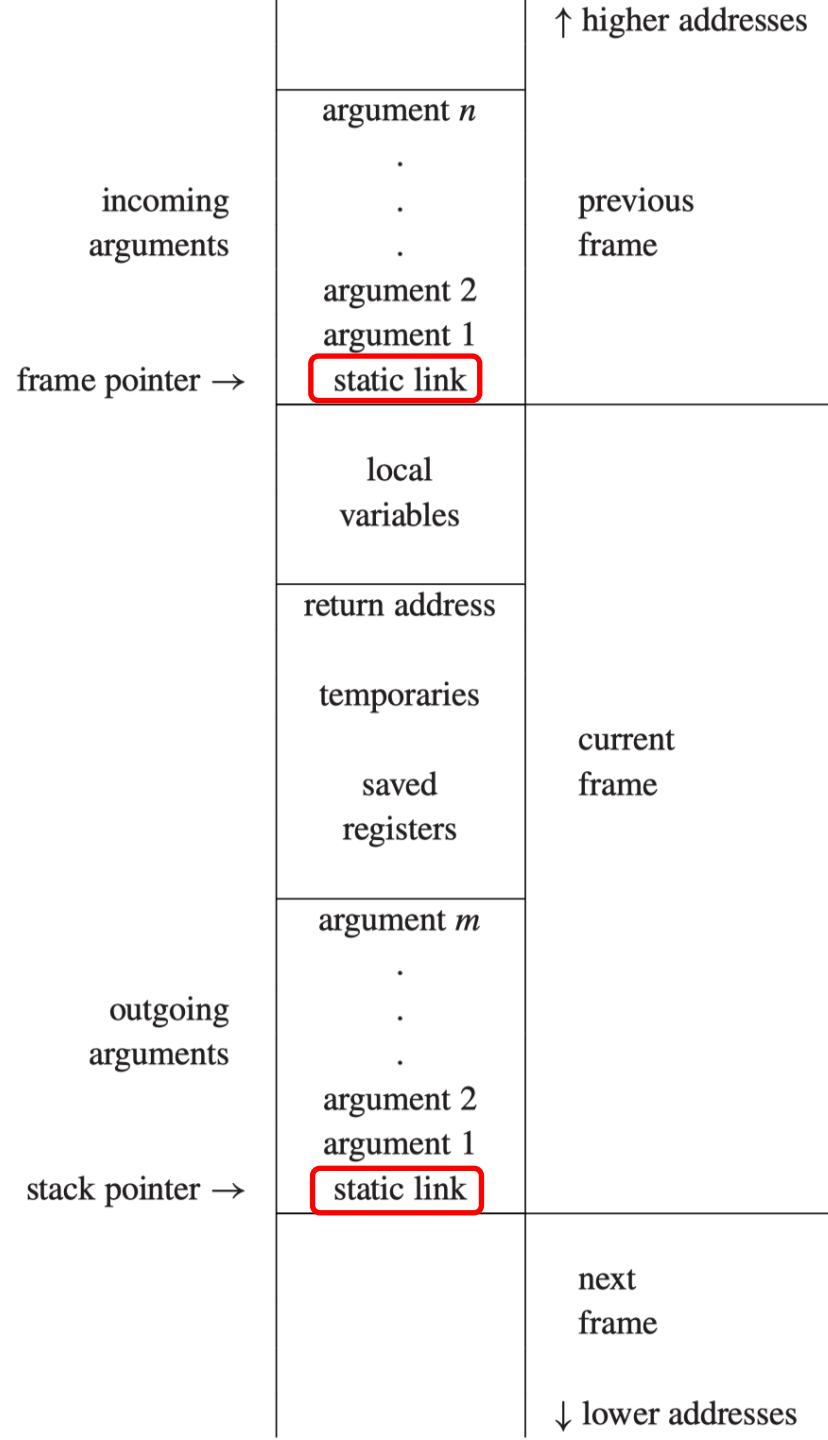
- the variable will be passed by reference, so it must have a memory address (e.g., `&` in the C language);
- the variable is accessed by a procedure nested inside the current one;
- the value is too big to fit into a single register;
- the variable is an `array`, for which address arithmetic is necessary to extract components;
- the register holding the variable is needed for a specific purpose, such as `parameter passing` (as described above);
- or there are so many local variables and temporary values that they won't all fit in registers, in which case some of them are “spilled” into the frame.

Frame-Resident Variables

- A variable *escapes* if :
 - it is passed by reference,
 - its address is taken (e.g., using C's *&* operator),
 - or it is accessed from a nested function
- E.g., **pas—by-reference** (supported in Pascal, but **not in Tiger**)
 - Locations of the actuals are passed;
 - References to the formals include implicit indirection to access values of the actuals.
 - Modifications to formals do change actuals!

Summary

- Registers hold
 - some parameters
 - return address
 - return value
 - some local variables and temporaries
- Stack frame holds
 - variables passed by reference or have their address taken (&)
 - variables that are accessed by procedures nested within the current one
 - variables that are too large to fit into register file.
 - array variables (address arithmetic needed to access array elements)
 - spilled registers (too many local variables to fit into register file, so some must be stored in stack frame)



Static Links

- ***Block Structure***: In languages allowing nested function declarations (such as Pascal, ML and Tiger), the inner functions may use variables declared in outer functions.
- We can access local variables through the Frame Pointer
 - Note that the actual value of FP is unknown until runtime, but the each local-variable's offset to FP is known at compile time!
- **How can a nested function access a “non-local” variable?**

Static Links

```
type tree = {key: string, left: tree, right: tree}
function prettyprint(tree: tree) : string =
  let
    var output := ""
    function write(s: string) =
      output := concat(output, s)

    function show(n: int, t: tree) =
      let function indent(s: string) =
        (for i := 1 to n
         do write(" "));
        output := concat(output, s);
        write("\n")
      in if t=nil
        then indent(".")
        else (indent(t.key));
            show(n+1, t.left);
            show(n+1, t.right)
      end
    in show(0, tree); output
end
```

How to make this work?

write must have access to the frame of *prettyprint*

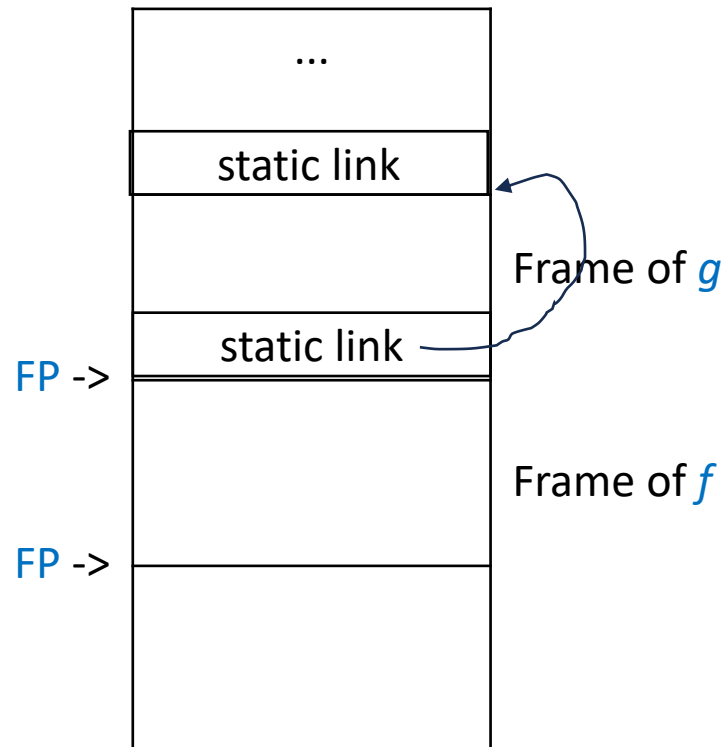
indent must have access to the frames of *show* and *prettyprint*

Static Links

- One solution: using **static links**

Whenever a function *f* is called, it is passed a pointer to the most recent activation record of the function *g* that **immediately encloses** *f* in the program text. This pointer is the **static link**.

```
int g (int x) {  
    int f(int y) {  
        ...  
    }  
    return f(x)+1;  
}
```



Static Links

```
type tree = {key: string, left: tree, right: tree}
function prettyprint(tree: tree) : string =
  let
    var output := ""
    function write(s: string) =
      output := concat(output, s)

    function show(n: int, t: tree) =
      let function indent(s: string) =
        (for i := 1 to n
         do write(" "));
        output := concat(output, s);
        write("\n")
      in if t=nil
        then indent(".")
        else (indent(t.key);
              show(n+1, t.left);
              show(n+1, t.right))
      end
    in show(0, tree); output
end
```

passing *prettyprint*'s own frame pointer as *show*'s static link:
prettyprint is the immediate enclosing function of *show*

show's static link = *prettyprint*'s frame pointer

Static Links

```
type tree = {key: string, left: tree, right: tree}
function prettyprint(tree: tree) : string =
  let
    var output := ""
    function write(s: string) =
      output := concat(output, s)

    function show(n: int, t: tree) =
      let function indent(s: string) =
        (for i := 1 to n
         do write(" "));
        output := concat(output, s);
        write("\n")
      in if t=nil
        then indent(".")
        else (indent(t.key));
             show(n+1, t.left);
             show(n+1, t.right))
      end
    in show(0, tree); output
end
```

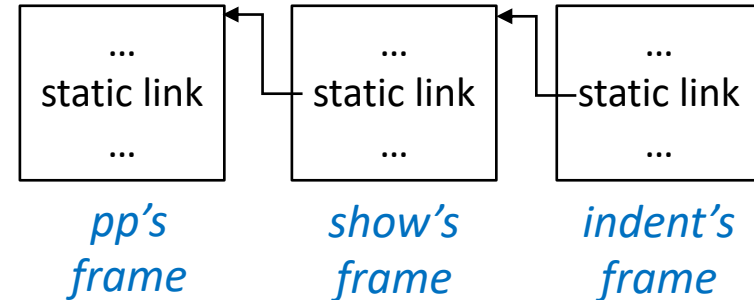
show calls *show*, passing its own **static link** (*not its own frame pointer*) as the static link



Static Links

```
type tree = {key: string, left: tree, right: tree}
function prettyprint(tree: tree) : string =
  let
    var output := ""
    function write(s: string) =
      output := concat(output, s)

    function show(n: int, t: tree) =
      let function indent(s: string) =
        (for i := 1 to n
         do write(" "));
        output := concat(output, s);
        write("\n")
      in if t=nil
        then indent(".")
        else (indent(t.key);
              show(n+1, t.left);
              show(n+1, t.right))
      end
    in show(0, tree); output
end
```



How can *indent* use *output* from *prettyprint*'s frame?

It starts with its own static link, then fetch *show*'s, then fetches *output*.

Static Links: Example

```
int f (int x, int y)
{
    int m;
    int g (int z)
    {
        int h () {
            return m+z;
        }
        return 1;
    }
    return 0;
}
```

```
int f (link, int x, int y)
{
    int m;
    int g (link, int z)
    {
        int h (link) {
            return link->prev->m +
link->z;
        }
        return 1;
    }
    return 0;
}
```

Static Links

- Using static links to access non-local variables
 - Each function is annotated with its enclosing depth
 - When a function at depth n accesses a variable at depth m
 - Emit code to climb up $n-m$ links to visit the appropriate activation record
- **Pros:**
 - Little extra overhead on parameter passing
- **Cons:**
 - The overhead to climb up a static link chain to access non-locals
 - Need a chain of indirect memory references for each variable access
 - Number of indirect references = difference in nesting depth between variable declaration function and use function
 - Functions may be deeply nested!

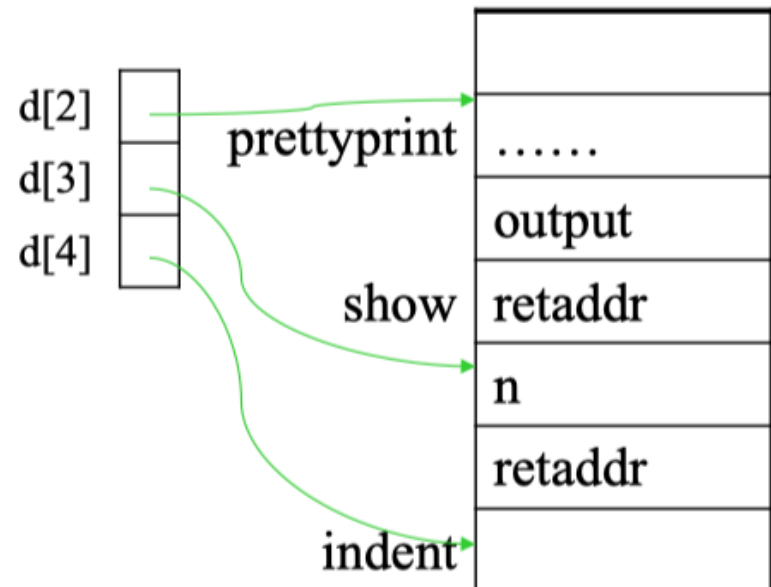
Display

There are several methods to implement **Block Structure**

- **Static link**
- **Display**: a global array of pointers to frames
 - Each function is annotated with its static nesting depth
 - Position *i* of this array is a pointer to the frame of the most recently entered procedure whose static nesting depth is *i*.

main	1
prettyprint	2
write	3
show	3
indent	4

Static nesting depths



Lambda Lifting

There are several methods to implement **Block Structure**

- **Static link**
- **Display**: a global array of pointers to frames
 - Each function is annotated with its static nesting depth
 - Position i of this array is a pointer to the frame of the most recently entered procedure whose static nesting depth is i .
- **Lambda lifting**: When g calls f , each variable of g that is actually accessed by f (or by any function nested inside f) is passed to f as an extra argument.
 - Rewriting the program by treating non-local variable as formal parameters.
 - The translation/transformation process starts with the innermost procedures and works its way outwards

Lambda Lifting: Example

```
int f (int x, int y)
{
    int m;
    int g (int z)
    {
        int h () {
            return m+z;
        }
        return 1;
    }
    return 0;
}
```

```
int f (int x, int y)
{
    int m;
    int g (int &m, int z)
    {
        int h (int &m, int &z) {
            return m+z;
        }
        return 1;
    }
    return 0;
}
```

Higher-Order Functions

- Using *stack*: we assume local variables will not be used after *f* returns
- In languages supporting both *nested functions* and *function-valued variables*, it may be necessary to keep local variables after a function has returned.

```
fun f(x) =  
  let fun g(y) = x + y  
  in g  
end  
  
val h = f(3)  
val j = f(4)  
  
val z = h(5)  
val w = j(7)
```

Written in ML

```
int (*)( ) f (int x) {  
  int g(int y) {  
    return x + y;  
  }  
  return g;  
  
int (*h)( ) = f(3);  
int (*j)( ) = f(4);  
int z = h(5);  
int w = j(7);
```

Written in pseudo-C

- The combination of *nested functions* and *functions returned as results*.
- Local variables need lifetimes longer than their enclosing function invocations

Higher-Order Functions

- **Pascal** has nested functions, but it does not have functions as returnable values.
- **C** has functions as returnable values, but not nested functions.
- **Pascal** and **C** can use stacks to hold local variables.
- **ML, Scheme, and several other languages** have both nested functions and functions as returnable values (**higher-order functions**).
 - They cannot use stacks to hold all local variables.

6.2 Frames in The Tiger Compiler

6.2 Frames in The Tiger Compiler

- The frame interface will look something like this:

```
/* frame.h */
typedef struct F_frame_ *F_frame;
typedef struct F_access_ *F_access;
typedef struct F_accessList_ *F_accessList;
struct F_accessList_ {F_access head; F_accessList tail;};
F_frame F_newFrame(Temp_label name, U_boolList formals);
Temp_label F_name(F_frame f);
F_accessList F_formals(F_frame f);
F_access F_allocLocal(F_frame f, bool escape);
```

- The abstract interface *frame.h* is implemented by a module specific to the target machine. E.g., *mipsframe.c*

```
/* mipsframe.c */
#include "frame.h"
...
```

Frames in The Tiger Compiler

```
/* frame.h */  
typedef struct F_frame_ *F_frame;  
F_frame F_newFrame(Temp_label name, U_boolList formals);  
Temp_label F_name(F_frame f);
```

- The type *F_frame* holds information about formal parameters and local variables allocated in this frame
- *F_newFrame(f, l)*: make a new frame for a function *f* with *k* formal parameters.
 - *l*: a list of *k* booleans: *true* for each parameter that escapes

Frames in The Tiger Compiler

```
/* frame.h */  
typedef struct F_access_ *F_access;
```

- The *F_access* type describes formals and locals that may be in the frame or in registers.
 - An abstract data type. The contents of *struct F_access_* are visible only inside the *Frame* module:
- e.g., *InFrame(X)*, *InReg(t₈₄)*

```
/* mipsframe.c */  
#include "frame.h"  
struct F_access_ {  
    enum {inFrame, inReg} kind;  
    union {  
        int offset; /* InFrame */  
        Temp_temp reg; /* InReg */  
    } u;  
};  
static F_access InFrame(int offset);  
static F_access InReg(Temp_temp reg);
```

Frames in The Tiger Compiler

```
F_accessList F_formals(F_frame f);
```

- The *F_formals* interface function extracts a list of *k* “accesses” denoting the locations where the formal parameters will be kept at run time, as seen from inside the callee.
- Parameters may be seen differently by the caller and the callee.
 - parameters are passed on the stack
 - *caller*: offset from the *stack pointer*
 - *callee*: offset from the *frame pointer*
 - parameters are passed through registers, e.g.,
 - *caller*: register 6
 - *callee*: register 13
- “Shift of View”

Frames in The Tiger Compiler

- This “**shift of view**” depends on the calling conventions of the **target machine**.
- It must be handled by the *Frame* module, starting with *newFrame*.
- For each formal parameter, *newFrame* must calculate two things:
 - How the parameter will be seen from inside the function (in a register, or in a frame location);
 - What instructions must be produced to implement the “**view shift**.”

Representation of Frame Descriptions

- The implementation module *Frame* is supposed to keep the representation of the *F_frame* type secret from any clients of the *Frame* module.
- *F_frame* is a data structure holding:
 - The locations of all the formals,
 - Instructions required to implement the “view shift,”
 - The number of locals allocated so far,
 - The label at which the function’s machine code is to begin

Representation of Frame Descriptions

- Suppose function *g* has three parameters with the first parameter escapes

		Pentium	MIPS	Sparc
Formals	1	InFrame(8)	InFrame(0)	InFrame(68)
	2	InFrame(12)	InReg(t_{157})	InReg(t_{157})
	3	InFrame(16)	InReg(t_{158})	InReg(t_{158})
View Shift		$M[sp + 0] \leftarrow fp$	$sp \leftarrow sp - K$	save %sp, -K, %sp
		$fp \leftarrow sp$	$M[sp + K + 0] \leftarrow r2$	$M[fp + 68] \leftarrow i0$
		$sp \leftarrow sp - K$	$t_{157} \leftarrow r4$	$t_{157} \leftarrow i1$
			$t_{158} \leftarrow r5$	$t_{158} \leftarrow i2$

- Why move *r4* and *r5* to t_{157} and t_{158} ?

```
function m(x:int, y:int) = (h(y,y); h(x,x))
```

- The register allocator will eventually choose which machine register should hold t_{157} and t_{158}

Local Variables

- Some local variables are kept *in the frame*; others are kept *in registers*.
- To allocate a new local variable in a frame *f*, the semantic analysis phase calls

```
F_access F_allocLocal(F_frame f, bool escape);
```

- If *escape = True*, *F_allocLocal* returns an *InFrame* access
- If *escape = False*, *F_allocLocal* can return an *InReg* access

Local Variables

```
function f() =  
  let var v := 6  
  in (print(v);  
      let var v := 7  
      in print(v)  
    end;  
  print(v);  
  let var v := 8  
  in print(v)  
  end;  
  print(v))  
end
```

```
void f() {  
  int v = 6;  
  print(v);  
  {int v = 7;  
   print(v);}  
  print(v);  
  {int v = 8;  
   print(v);}  
  print(v);  
}
```

- There may be variable-declaration blocks nested inside the body of a function.
- Three different variables
- **What is the result?**
- print 6 7 6 8 6
- When to call *F_allocLocal*?

- As each variable declaration is encountered during processing, *allocLocal* will be called to allocate a temporary or new space in the frame, associated with the name *v*.
- As each *end* (or closing brace) is encountered, the association with *v* will be forgotten—but the space is still reserved in the frame.
- A distinct temporary or frame slot for every variable declared within the entire function.

Local Variables

```
function f() =  
  let var v := 6  
  in (print(v);  
      let var v := 7  
      in print(v)  
    end;  
    print(v);  
    let var v := 8  
    in print(v)  
  end;  
  print(v))  
end
```

```
void f() {  
  int v = 6;  
  print(v);  
  {int v = 7;  
   print(v);}  
  print(v);  
  {int v = 8;  
   print(v);}  
  print(v);  
}
```

- Three different variables
- print 6 7 6 8 6

- The register allocator will use **as few registers as possible** to represent the temporaries.
 - The second and third **v** variables could be held in the same temporary
- A clever compiler might also notice two frame-resident variables could be allocated to the same slot.

Calculating Escapes

- Local variables that do not escape can be allocated in a register; escaping variables must be allocated in the frame.
- When calling *F_allocLocal*, it is important to know whether the variable escapes or not.
- The *findEscape* function:
 - Traverse the entire abstract syntax tree, looking for escaping uses of every variable.
 - Use environments to record whether the particular variable escapes.

```
/* escape.h */
void Esc_findEscape(A_exp exp);
/* escape.c */
static void traverseExp(S_table env, int depth, A_exp e);
static void traverseDec(S_table env, int depth, A_dec d);
static void traverseVar(S_table env, int depth, A_var v);
```

Temporaries and Labels

- The compiler's semantic analysis phase will want to **choose registers** for parameters and local variables, and **choose machine-code addresses** for procedure bodies.
- But it is too early to determine them exactly.
- Idea: use “abstract registers” and “abstract addresses”
- **Temporary**: a value that is temporarily held in a register
- **Label**: some machine-language location whose exact address is yet to be determined

Temporaries and Labels

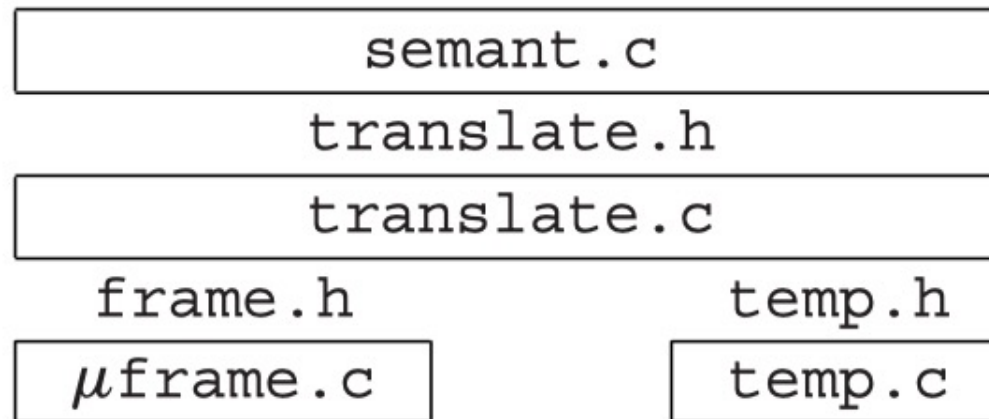
- *Temps*: abstract names for local variables
- *Labels*: abstract names for static memory addresses

```
/* temp.h */
typedef struct Temp_temp_ *Temp_temp;
Temp_temp Temp_newtemp(void);

typedef S_symbol Temp_label;
Temp_label Temp_newlabel(void);
Temp_label Temp_namedlabel(string name);
string Temp_labelstring(Temp_label s);

typedef struct Temp_tempList_ *Temp_tempList;
struct Temp_tempList_ {Temp_temp head; Temp_tempList tail;}
Temp_tempList Temp_TempList(Temp_temp head, Temp_tempList
tail);
typedef struct Temp_labelList_ *Temp_labelList;
struct Temp_labelList_ {Temp_label head; Temp_labelList tail;}
Temp_labelList Temp_LabelList(Temp_label head, Temp_labelList
tail);
```

Two Layers of Abstraction



- The Tiger compiler will have two layers of abstraction between semantic analysis and frame-layout details:
- The *frame.h* and *temp.h* interfaces provide machine-independent views of memory-resident and register-resident variables.
 - We need not care where variables are exactly stored.
- The **Translate** module augments this by handling the notion of **nested scopes (via static links)**, providing the interface **translate.h** to the Semant module.

Two Layers of Abstraction

```
/* translate.h */
typedef struct Tr_access_ *Tr_access;
typedef ... Tr_accessList ...
Tr_accessList Tr_AccessList(Tr_access head, Tr_accessList tail);
Tr_level Tr_outermost(void);
Tr_level Tr_newLevel(Tr_level parent, Temp_label name,
U_boolList formals);
Tr_accessList Tr_formals(Tr_level level);
Tr_access Tr_allocLocal(Tr_level level, bool escape);
```

- **Translate** manages local variables and static function nesting for **Semant**.
- Managing the *Tr_level* of each formal, local variable and function.

Two Layers of Abstraction

```
/* new versions of VarEntry and FunEntry */
struct E_entry_ {
    enum {E_varEntry, E_funEntry} kind;
    union {
        struct {Tr_access access; Ty_ty ty;} var;
        struct {Tr_level level; Temp_label label;
                Ty_tyList formals; Ty_ty result;} fun;
    } u;
};

E_entry E_VarEntry(Tr_access access, Ty_ty ty);
E_entry E_FunEntry(Tr_level level, Temp_label label,
Ty_tyList formals, Ty_ty result);
```

```
/* inside translate.c */
struct Tr_access_ {Tr_level level; F_access access};
```

- The *Tr_level* of each function or variable is stored in the environments.

Managing Static Links

- We use the **Translate** module to manage static links.
- Why not use the **Frame** module to manage them?
 - Many source languages do not have nested function declarations.
 - **Frame** should be independent of the specific source language being compiled
- **Translate** knows that each frame contains a static link. The static link is **passed to a function in a register** and **stored into the frame**.
 - just like a parameter
- We will **treat the static link as a parameter** (as much as possible)

Keeping Track of Levels

```
/* translate.h */
typedef struct Tr_access_ *Tr_access;
typedef ... Tr_accessList ...
Tr_accessList Tr_AccessList(Tr_access head, Tr_accessList tail);
Tr_level Tr_outermost(void);
Tr_level Tr_newLevel(Tr_level parent, Temp_label name,
U_boolList formals);
Tr_accessList Tr_formals(Tr_level level);
Tr_access Tr_allocLocal(Tr_level level, bool escape);
```

- *Tr_outermost*: returns the outermost level
- The level within which the Tiger main program is nested
- All “library” functions are declared at this outermost level, which does not contain a frame or formal parameter list.

Homework

- **6.3, 6.7(a,b)**

Temporaries and Labels

- *Temp_newtemp()* returns a new temporary from an infinite set of temps.

```
/* temp.h */
typedef struct Temp_temp_ *Temp_temp;
Temp_temp Temp_newtemp(void);

typedef S_symbol Temp_label;
Temp_label Temp_newlabel(void);
Temp_label Temp_namedlabel(string name);
string Temp_labelstring(Temp_label s);

typedef struct Temp_tempList_ *Temp_tempList;
struct Temp_tempList_ {Temp_temp head; Temp_tempList tail;}
Temp_tempList Temp_TempList(Temp_temp head, Temp_tempList
tail);
typedef struct Temp_labelList_ *Temp_labelList;
struct Temp_labelList_ {Temp_label head; Temp_labelList tail;}
Temp_labelList Temp_LabelList(Temp_label head, Temp_labelList
tail);
```

Temporaries and Labels

- *Temp_newlabel()* returns a new label from an infinite set of labels.

```
/* temp.h */
typedef struct Temp_temp_ *Temp_temp;
Temp_temp Temp_newtemp(void);

typedef S_symbol Temp_label;
Temp_label Temp_newlabel(void);
Temp_label Temp_namedlabel(string name);
string Temp_labelstring(Temp_label s);

typedef struct Temp_tempList_ *Temp_tempList;
struct Temp_tempList_ {Temp_temp head; Temp_tempList tail;}
Temp_tempList Temp_TempList(Temp_temp head, Temp_tempList
tail);
typedef struct Temp_labelList_ *Temp_labelList;
struct Temp_labelList_ {Temp_label head; Temp_labelList tail;}
Temp_labelList Temp_LabelList(Temp_label head, Temp_labelList
tail);
```

Temporaries and Labels

- *Temp_namedlabel(string)* returns a new label whose assembly-language name is string.

```
/* temp.h */
typedef struct Temp_temp_ *Temp_temp;
Temp_temp Temp_newtemp(void);

typedef S_symbol Temp_label;
Temp_label Temp_newlabel(void);
Temp_label Temp_namedlabel(string name);
string Temp_labelstring(Temp_label s);
```

```
typedef struct Temp_tempList_ *Temp_tempList;
struct Temp_tempList_ {Temp_temp head; Temp_tempList tail;}
Temp_tempList Temp_TempList(Temp_temp head, Temp_tempList
tail);
typedef struct Temp_labelList_ *Temp_labelList;
struct Temp_labelList_ {Temp_label head; Temp_labelList tail;}
Temp_labelList Temp_LabelList(Temp_label head, Temp_labelList
tail);
```

Be careful: there could be different functions with the same name in different scopes

Two Layers of Abstraction

```
/* translate.h */
typedef struct Tr_access_ *Tr_access;
typedef ... Tr_accessList ...
Tr_accessList Tr_AccessList(Tr_access head, Tr_accessList tail);
Tr_level Tr_outermost(void);
Tr_level Tr_newLevel(Tr_level parent, Temp_label name,
U_boolList formals);
Tr_accessList Tr_formals(Tr_level level);
Tr_access Tr_allocLocal(Tr_level level, bool escape);
```

- *Tr_newLevel*: creates a new “nesting level” for each function, called by *transDec*
- *Tr_allocLocal(lev, esc)*: When *Semant* processes a local variable declaration at level *lev*, it calls *Tr_allocLocal(lev, esc)* to create the variable in this level.

Managing Static Links

```
/* translate.h */
typedef struct Tr_access_ *Tr_access;
typedef ... Tr_accessList ...
Tr_accessList Tr_AccessList(Tr_access head, Tr_accessList tail);
...
Tr_accessList Tr_formals(Tr_level level);
...
```

- *Tr_formals*: When *Semant* calls *Tr_formals(level)*, it will get the *access* values of the original parameters.